

HIGH LEVEL LANGUAGES AND THE INTERPRETER

Since the start of the computer era, there has been the need to express problems for the computer in a form that is understandable to humans, yet easily mechanized for execution in a processor. Early systems placed the burden of effort wholly on the programmer, forcing them to structure their problems in steps that corresponded one-for-one with the basic machine operations. One can only imagine the difficulty in translating symbolic mathematical equations into the arcane language of basic arithmetic and procedural operations. In the section on AGC instructions, Figure 24 showed an example of using basic instructions to add a list of numbers. While quite useful as a demonstration of several interesting AGC instructions, it also showed the level of detail necessary for performing even trivial tasks. Expending a large amount of effort to develop such a routine is acceptable if the code remains static. However, if there is a change in the AGC instruction architecture, or if the routine is ported to an entirely new architecture, we are forced to rewrite the code to reflect the new hardware characteristics. The time and expense of such a recoding effort is often so high that it negates using a newer, more flexible architecture. A not completely implausible example would be implementing the routine to sum a table of numbers in the Space Shuttle computer. The Shuttle's General Purpose Computers are a completely different design than the AGC, so our program must be rewritten to reflect the different architecture. Figure 44 shows how this might look in the computer's native, low-level language.¹⁴

While the Space Shuttle code is arguably more compact and efficient, there is still the issue of rewriting the summation program to use multiple registers, indexes and a

	SR	R2,R2	SET SUM IN REGISTER 2 TO ZERO
	SR	R3,R3	STARTING INDEX IS ZERO
	LA	R4,4	INCREMENT IS 4 BYTES IN THE TABLE
	LA	R5,20	FIVE TABLE ENTRIES * 4 BYTES/ENTRY
LOOP	A	R2,NUMBRTAB(R3)	ADD THE NEXT ENTRY IN THE TABLE
	BXLE	R3,R4,LOOP	INCREMENT THE INDEX, CHECK IF
*			WE ARE DONE, IF NOT, CONTINUE
NUMBRTAB	DC	'2'	
	DC	'5'	
	DC	'7'	
	DC	'4'	
	DC	'6'	

Figure 44: Summing a table of numbers in the Space Shuttle computer

¹⁴ Some readers will recognize this code as IBM 360 assembler. The Shuttle General Purpose Computer, the AP 101S, is based on the classic IBM System 360 architecture. Although the AP 101S is not completely compatible with the S/360 series, it is sufficiently similar that using S/360 assembly for this example is acceptable.

complex branch instruction. Clearly, the goal is to express the problem in a form that is as independent of the underlying architecture as possible. At the same time, the notation should more closely reflect how the user would express the original problem. High level languages are a huge improvement over machine level coding, as they reflect the problem statement more closely, and are far easier to read and maintain. Literally hundreds of languages exist, each with its own target audience, but some of the most familiar are C, Java, and yes, even Cobol. The power of a high level language is easily demonstrated by the trivial assignment of the value “1” to the variable “X”. Coding this in the AGC might look like this:

	CA	ONE	COPY THE VALUE OF 1 INTO ACCUMULATOR
	TS	X	SAVE THE VALUE OF 1 IN STORAGE
	...		
X	DEC	0	STORAGE AREA CALLED “X”
ONE	DEC	1	VALUE OF ONE

The same expression can be coded in the C language:

```
integer x = 0;
x = 1;
```

While this small code fragment is not a particularly dramatic improvement over the AGC code, mathematicians will immediately understand the intent. We now expand the idea of high level coding, using our number summation program written in the C language:

```
integer numbrtab[5] = {0,1,6,4,2,3};
integer sum = 0;
integer i = 0;
for (i=0; i++ < 5) {
    sum = sum + numbrtab[i];
}
```

While not completely intuitive, this code is far more readable than our attempts with machine coding, and at first glance appears to be the solution to our coding needs. While the program is more “human friendly”, it cannot execute in its current form without a conversion to the processor’s native machine instructions. A software program known as a compiler performs this task, taking each of the high level statements and generating the equivalent machine instructions. Unfortunately, there are important limitations to the quality of code that a compiler can generate. In particular, early compilers did not produce efficient code, compared with the best hand-written machine instructions. The fact that both execution and memory requirements are larger for compiled programs, made high level languages impractical in the resource-limited AGC.

A compromise between the high efficiency of machine code and the easy programmability of a high level language is realized in the AGC’s Interpreter. As implemented in modern computers, interpreters perform a similar function to that of a compiler. Both begin with the source code written as a high level language. After

taking the source statements and generating machine instructions, a compiler will never need the original source code again. Interpreters do not generate machine level instructions for the hardware to execute, but use a different process to convert high level statements into a form that can be processed. The simplest interpreters begin with the original source code, and execute the program directly without converting the program into a more efficient representation. Each source statement is reparsed and reinterpreted each time it is encountered, regardless of how many times the statement has been executed previously. As expected, this approach is very slow and inefficient, and there are no intermediate results from the interpretive process to save and reuse.

The output of more sophisticated interpreters is not machine instructions as produced by a compiler, but an intermediate language, often called “pseudo-code” or “P-code”. This interpretive code is the “language” of the interpretive system, and is analogous to a processor’s machine instructions. An imaginary, or “virtual”, processor is written in software, and it reads and executes the pseudo-code as its “basic instructions”. A huge benefit of such a virtual machine is the ability to create any desired processor characteristic, bypassing the limitations inherent in the underlying hardware’s architecture. In the description of the basic design of the AGC, the lack of support for index registers and sophisticated datatypes is apparent. With an interpreted system, many of the limitations of the AGC’s architecture are neatly eliminated. Interpreters are targeted for developing applications such as the mission programs, rather than system-level coding in the Executive. Restricting the interpretive software to mission software eliminates the complexities involved with managing hardware operations such as interrupt handling. This is usually not a problem, since Executive programs rarely use complex datatypes or complex indexing, and cannot afford the greatly increased execution time that interpreted programs require.

Unlike contemporary languages like Java, whose statements are converted to “P-code” and then executed, there is no comparable high-level language for the AGC. The task of efficiently converting human readable code to an internal format requires sophisticated optimizing compilers and interpreters, which are fiendishly difficult to develop, and were simply not available to the AGC developers. To create the most efficient programs possible, the AGC developers wrote the intermediate code directly, bypassing the troublesome conversion from a high level language.

Parsing high level language statements

To understand the AGC’s interpretive system, it is essential to return to a fundamental issue. What is the process of converting the high level logic and equations into executable machine instructions? Beginning with a simple equation:

$$a = \frac{(b^2 + c^2)}{(b^2 - c^2)}$$

The limitations of typing on a computer require equations to be coded on a single line:

$$a = (b^{**2} + c^{**2}) / (b^{**2} - c^{**2})$$

While perhaps not as graphically satisfying as our original expression, the new representation is equivalent and has its own advantages. The sequence of operations is more apparent, using the convention of reading from left to right. Parentheses are used to group operations together and eliminate any ambiguity in identifying which terms should be executed first. Because superscripts are not possible in this simple single-line notation, exponents are replaced by the characters “**”. Using this notation, b^2 is now rewritten as $b**2$. Starting with the expression immediately following the equals sign, the value of b is squared, as is the value of c , and the two are added together. Now that the numerator has been evaluated, it is time to begin work on the denominator. However, both the numerator and denominator must be evaluated before the division operation can proceed. To preserve the work already done on the numerator, the intermediate value of $(b^2 + c^2)$ is set aside in a temporary location that we will call T1. Next, the value of c^2 is subtracted from the value of b^2 . Like the first half of the equation, this intermediate value is saved as well, this time in a location called T2. The only remaining operation is to divide the temporary value of T1 by the temporary value T2, and save the result in the variable named a .

Processing an equation requires reading and parsing the entire statement, and analyzing the result to determine the proper execution sequence. The notation used has its roots in computer science, and is called a “tree structure”. Many problems in computer science lend themselves to being expressed as trees, especially those with complex interactions. In its simplest form, a tree structure has “branches” and “leaves”, akin to its physical namesake. Operands such as variables are the leaves, are connected to branches, and are joined by a single computer operation. In addition to benefiting the programmer with a graphical description of the problem, translating a tree structure into basic computer instructions is quite straightforward. An expression, $3 + 2 = 5$, has two numbers added together to create a sum. In a tree structure, this expression is described in Figure 45(a). Read from the bottom-most leaves upwards, we take the two operands, three and two, and add them together to create five as the sum. Extending this form to an expression such as $X = 3 + 2$, we have the variable X assigned the sum of three plus two in Figure 45(b); and 45(c) creates a generalized expression which retains the same structure, yet now is usable for any set of input variables.

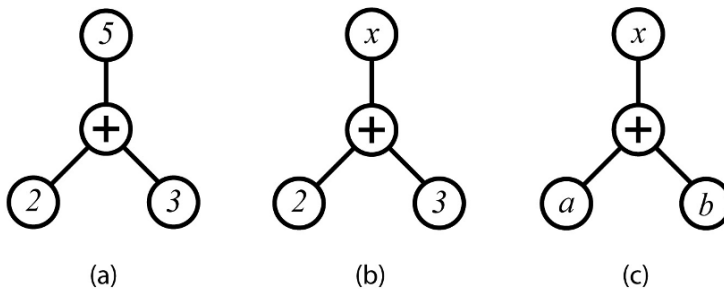


Figure 45: Expressions in tree structures

The full range of binary operations are available to the programmer, including addition, subtraction, multiplication and division. Unary operators also exist, and include not only negation, but also mathematical operations such as square and root plus trigonometric functions.

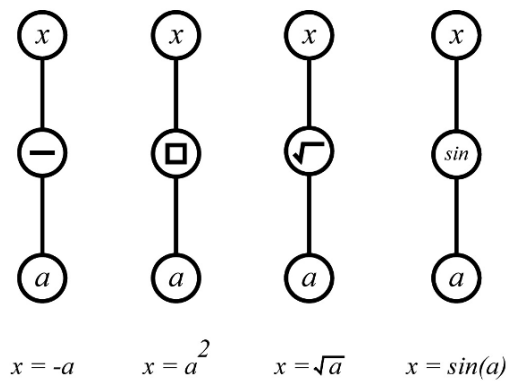


Figure 46: Unary and trigonometric functions

As with all building blocks, each such individual element appears barely worthy of even passing attention. It is only when they are combined that they become truly interesting. Consider a slightly more complex expression, $E = mc^2$. This familiar equation requires both a unary and a binary operation. Traversing the tree upwards, the first step is to square the value of c . The result is multiplied by m , and the final value placed in E .

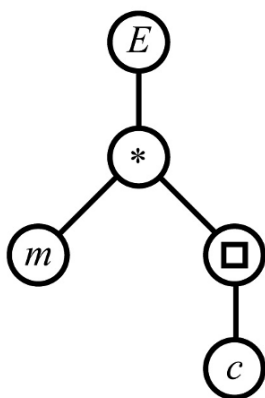


Figure 47: Tree representing $E = mc^2$

The ability to assemble basic operations into tree structures to define an equation is now becoming apparent. A final, more complex example allows us to address the more important issues that arise in real world problems. Consider the polynomial expression $ax^2 + bx + c = 0$. Solving for the value of x requires the quadratic equation:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Two values for x are possible, so we must break it into two separate expressions.

$$x = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \text{ and } x = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

The trees differ only by the plus or minus term in the numerator, so our next example works equally well with either expression. A tree structure of the first equation looks like:

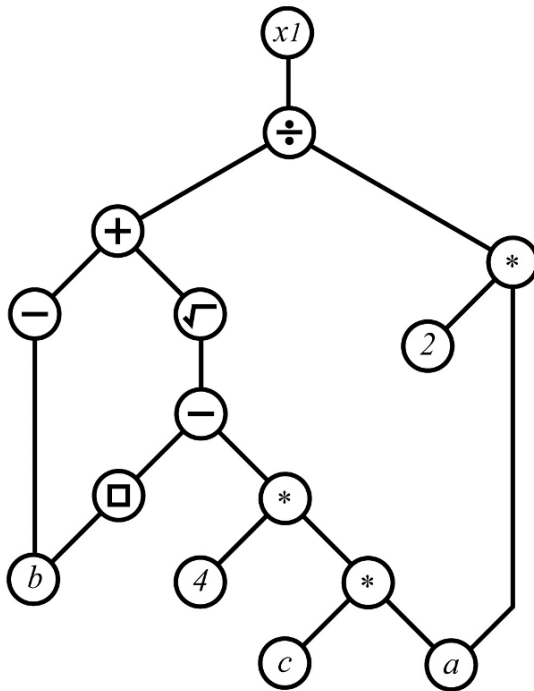


Figure 48: Quadratic equation tree

The quadratic equation uses two variables (a and b) more than once, which complicates our tree somewhat. The variable b is both squared and negated, and a is in both the numerator and denominator. The equation is already in its simplest form, so it is not possible to combine or cancel out either variable.

Traversing the quadratic equation

We return to one of the possible roots of the quadratic equation:

$$x = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

The traversing process starts in the lower left-hand corner of the tree, with the goal of traversing all the branches to the right and upwards. Only a few rules are necessary for navigating the tree, and they reflect the natural process of evaluating an expression from left to right.

- Start at the lowest, left-most leaf on the tree. If there are multiple branches on a leaf, start with the left-most branch.
- Variables, constants and temporary variables are all handled in the same manner.
- A branch is traversed only once.
- All functions leave their results in the accumulator.
- Upon reaching a function by going up one of its branches, take the other branch down before continuing back up. Continue going down the tree until a leaf is found.
- When a leaf is found, go up one level and execute that function.
- If two leaves are found underneath a function, select the left-hand leaf first, then the right one.
- When traversing up to a function that has another untraversed branch, look ahead to see if that branch contains another function. If it does, and we have arrived at the function through the left-hand branch, save the contents of the accumulator in a unique temporary variable. Once saved, the accumulator is cleared to zero. Otherwise, the intermediate values already calculated up to this point would be lost when traversing to the other branch.
- Operations are performed only when going up branches on the tree, never down.

The process of traversing the quadratic equation tree uses the following steps:

- 1) Start with the variable leaf b.
- 2) Negate the value of b.
- 3) Addition has two operands. Looking down the tree, there are two branches underneath this function, and both are functions themselves. Coming up from the left branch, the accumulator must be saved in a temporary variable that we will call T1. Next, the accumulator is cleared to prevent interference with the next set of arithmetic instructions. The next branch is down and to the right, to the Square Root function.

- 4) Square Root is a unary function. No special handling is necessary, and we continue down to determine its operand.
- 5) Subtraction, like addition, requires two operands. Both are functions, requiring that the intermediate variable be saved when we finish traversing the left branch.
- 6) The first function is Squaring. Like Square Root, Square is a unary operation, so we continue down the next branch and see the variable b . This is squared, placed in the accumulator, and we move up the left branch of the Subtract function.
- 7) Back at the Subtract function, all of the operations on the left-hand branch are completed. Noticing that the right-hand branch is another function, we will again save the accumulator, this time in temporary location T2.
- 8) Moving down the right-hand branch of the Subtract, we encounter the Multiply operation. On the left branch we have the constant 4, so no further progress is possible here. The branch to the right will provide the second operand.
- 9) The branch takes us to another Multiplication function, with its two branches leading to the variables a and c . There is now enough information to perform the Multiply, so the product of a and c is placed in the accumulator.

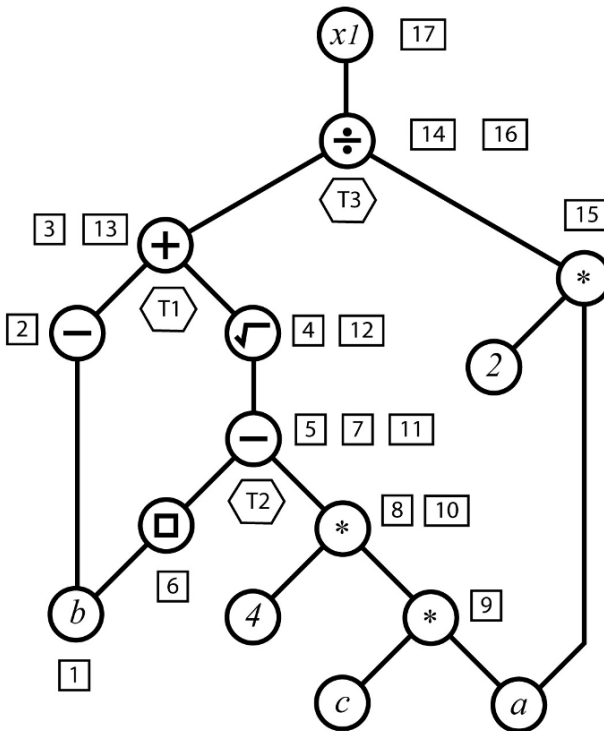


Figure 49: Labeled quadratic equation tree

- 10) Moving up with a value in the accumulator, the next step is to perform the Multiplication of the accumulator and the constant 4. Once the Multiply is complete, the traverse up the tree can continue.
- 11) Returning to the Subtraction operation, there is the left-hand value stored in the temporary value T2, and the value left in the accumulator from the just completed traverse of the right-hand branch. The contents of the accumulator are subtracted from T2, the result saved in the accumulator, and the traverse continues upwards.
- 12) Back to the Square Root, now with a value that that is usable for that function. Take the Root of the accumulator contents, replacing the existing value in the accumulator.
- 13) We now find ourselves in a similar situation as we were in step 11. The value for the left-hand side (-b) is saved in temporary variable T1. The addition operation takes T1 and adds it to the accumulator.
- 14) Calculations for the numerator are now complete, and attention moves to the denominator of the equation. The division operation has a value provided through its left-hand branch, but the right-hand side is a Multiplication function. We will save the intermediate result in another temporary variable, T3, and begin the traverse down the right-hand branch. Again, the accumulator is cleared when creating the temporary variable.
- 15) The Multiplication has the two variables available as leaves on the tree, so we can perform the Multiplication and put the result in the accumulator.
- 16) The Division now has both variables. The numerator from the left branch in variable T3. This is divided by the contents of the accumulator built from the operations on the right branch. Since the denominator is in the accumulator, we need to exchange it with T3 before the Division takes place.
- 17) We're finished! All the arithmetic operations are complete, and the final value is saved in variable X1.

Stacks and temporary variables

This admittedly elaborate example is useful for showing several key concepts of how equations are processed by an interpreter and executed. Parsing and generating the tree structure is the most familiar and practical methodology to represent the left-to-right evaluation of a mathematical statement. With only a single accumulator, the limitations of the AGC become apparent. Finding ourselves with intermediate values that are not usable immediately, saving the accumulator in a temporary location is necessary. In managing these temporary variables, a few surprising properties of a tree structure become apparent. First, an intermediate value needs to be saved only once, and needs to be retrieved only once as well. Although there may be many intermediate values saved at one particular time, a pattern emerges in the sequence of storing and retrieving the data. The most recently saved temporary data is always the first brought back into the equation. Any data saved previously is recalled only after more recently saved

data is itself retrieved, a process called “first in, last out”. To see this property demonstrated, assume three intermediate values T1, T2 and T3 are saved in sequence: T1 is saved first, then T2 and lastly T3. Only after the point where T3 is retrieved will T2 be needed for its part of the equation. Continuing further, only after T2 is used will T1 become available in the expression.

A restaurant kitchen offers a perfect physical analogy of how the temporary variables are referenced. Plates are stacked in a dispenser cart, which contains a spring to keep the top plate visible and easy to grasp. Whenever a clean plate arrives from the dishwasher, it is placed on top of the stack, pushing down whatever other plates might already be in the dispenser. When the chef needs a new plate, she takes it from the top of the stack and the remaining plates “pop up”. With this analogy, we have described the most common and powerful data structure in computer science, the stack. When data has to be stored in an ordered sequence and retrieved in reverse order, stacks are used to store and keep track of the data. The stack is especially appropriate where one program calls another, which in turn, calls a third, which calls a fourth, etc. Local variables and parameters are pushed onto the stack at each successive program call in the same manner that plates were placed onto the dispenser. As each program completes processing and control returns to its calling program, the data is “popped” from the stack.

A particularly important characteristic for the AGC is that a stack uses memory very efficiently. Consider the case if, instead of a stack, we explicitly assigned a word of memory for each temporary variable that we might use. Programs generally do not simultaneously use all temporary variables at all times; yet without a stack, all of the storage for temporary variables must be allocated in advance. Memory is wasted since only a few temporary variables are needed at any one time, leaving many words of storage sitting idle. Granted, it is possible to develop elaborate schemes to reuse specific memory areas, but these are cumbersome and error prone. In contrast, a stack “grows” in storage only as a program requires it.

Stack implementations follow the physical analogy of the plate dispenser only so far. Each time a plate is added or removed, the remaining plates physically move up or down in unison. In designing a software version of the dispenser, there is no need to move the data back and forth in memory. Rather, we reserve enough storage to hold the maximum amount of data that might be expected in the stack. The first item is placed in the first storage location in the stack memory area, and is (by definition) on the “top”. When the second item of data needs to be placed on the stack, the first data word is not moved out of the way. Instead, the new word is placed in the second storage location in the stack. This action creates a dilemma: specifically, how is it possible to keep track of the location where the “top” of the stack is? Solving this problem is done by introducing a new variable that is an integral part of a stack implementation, the stack pointer. Stack pointers, as their name implies, always reference the most recent item in the stack. “Pushing” a new item of data onto the stack requires that the stack pointer change its reference to the next available location, into which the data is copied. “Popping” data off the stack operates in the opposite way. After retrieving the data at the top of the stack and passing it to the requesting program, the stack

pointer is updated to reference the next oldest entry. Note that only the value of the stack pointer changes during the push and pop operations, and the data in the stack remains fixed. Clearing out the locations that previously held the “popped” data is not a mandatory housekeeping task, but it is often good practice to do so. All stack operations must use the stack pointer to reference the data, and only the top element is legitimately available. From this set of assumptions, any storage not within the scope of the stack at any given moment is irrelevant.

Double and triple precision datatypes

The notion of “datatypes”, that is, whether a variable is single or double precision, integer or fractional, has had little relevance until this time. In the task of analyzing an equation and creating a process tree, the exact characteristics of the datatypes were not important. Only now, as the process tree is converted into machine instructions do these characteristics need to be considered. The example for finding one root of a quadratic equation used single precision values, but double or even triple precision variables are possible simply by generating a few extra machine instructions for each arithmetic operation. Generating an extra instruction or two might be acceptable for multiple precision values, if done sparingly. However, when complex datatypes (such as vectors and matrices) are used, repeatedly generating the same instruction sequences consumes large amounts of memory. Using a traditional subroutine library for complex operations is one alternative, with parameter lists of input and output for exchanging data with the main program. Unfortunately, this approach also consumes memory at a prodigious rate, and incurs the overhead of moving data in and out of the parameter list.

At this point, extending the AGC architecture further only results in marginal functional improvements. Even if the new system extensions are desirable, the additional level of system complexity is expensive in terms of the development and testing effort.

Thus, we conclude that the AGC is ripe for a radical departure from its currently implemented architecture.

THE INTERPRETER

Thus far, this chapter has focused on the system level software found in the Executive. System routines are characterized by their own type of complexity and operate in a world of interrupts, timers and process management. Logically complex, the programming focus of the Executive software is hardware, and it uses simple datatypes such as single and double precision variables. While the Executive routines are essential, they are only the necessary overhead required for the most important software: the mission-specific programming. Mission software is very different from that in the Executive. Each mission program aims to achieve a specific objective, whether it be navigating to the Moon, the landing itself, or the rendezvous after the lunar module ascends from the surface.

Developing the mission software to solve these problem is a huge undertaking.